



ORBIS: AN INTELLIGENT ACADEMIC SUPPORT SYSTEM WITH AI AND RAG

by

ATAKAN GÜL, 121200152
ARDA KAAAN YILDIZ, 122200045

Supervised by

ÖZGÜR ÖZDEMİR

Submitted to the
Faculty of Engineering and Natural Sciences
in partial fulfillment of the requirements for the

Bachelor of Science

in the
Department of Computer Engineering

June, 2026

TABLE OF CONTENTS

TABLE OF CONTENTS	ii
1 Realistic Constraints	1
1.1 Design	1
1.2 Social, Environmental and Economic Impact within the Scope of the UN Sustainable Development Goals	2
1.3 Project Outcomes	3
1.4 Cost Analysis	3
1.5 Sustainability	4
1.6 Standards	5
1.6.1 Engineering Code Of Conduct	5
1.6.2 IEEE Standards	6
1.6.3 Software Quality, Security and Accessibility Standards	6
1.6.4 EU And Turkish Legal Standards	7
2 Risks	7
2.1 Risk Analysis	7
2.2 Potential Risks and B Plan	7
2.3 Risk and Change Management	8
2.4 Challenges	9
2.5 Weak And Strong Points	9
2.5.1 Weak Points	9
2.5.2 Strong Points	10
3 Contributions	10
4 Conclusion	11
References	12

1 Realistic Constraints

This report discusses the realistic constraints of Orbis. Report 1 already explains the technical architecture, methodology, experiments and results. Therefore, this document focuses on the social, environmental, economic, ethical, standardization, risk and contribution parts of the project.

Orbis is an academic support system that uses AI and Retrieval-Augmented Generation to detect academic obligations, match them with student profiles and help students follow university regulations. Since the project is related to education and student data, it must be evaluated not only as a software product but also as an engineering system that can affect real users.

1.1 Design

The design of Orbis was shaped by the realistic constraints of a senior engineering project: limited time, limited budget, no official production access to the Student Information System, and the need to avoid unsafe automated academic decisions. The system was therefore designed as an advisory and evidence-based support layer instead of a final authority.

The main design decision is to keep the system modular. The knowledge-base crawler, taxonomy generation pipeline, RAG search service, event extraction pipeline, assignment matcher, submission agent, database layer and frontend are separate components. This makes the project easier to test and maintain because a weak component can be improved without rewriting the whole system. For example, the assignment matcher can be replaced or re-trained while the regulation crawler and source-backed search interface remain usable.

The second design constraint is traceability. Orbis does not only produce an answer; it also stores source URLs, evidence text, extracted rules, reasoning logs and review states. This is important because an academic support system must allow students, advisors and administrators to inspect why a recommendation was generated. A black-box design would be faster to prototype, but it would not be acceptable for educational decision support.

The third design constraint is deployability within a university environment. The selected stack, FastAPI, PostgreSQL, `pgvector`, React and open-source Python tooling, can run locally or on a small server. This keeps the project within a student budget and avoids dependence on expensive proprietary infrastructure. The design also allows sensitive student data to be minimized

before any LLM call, and it can be moved to an on-premise model if institutional policy requires it.

1.2 Social, Environmental and Economic Impact within the Scope of the UN Sustainable Development Goals

For an engineering project to be successful, it should create a useful effect for people or institutions. Orbis mainly creates a social impact by helping students understand academic rules earlier and more clearly. University regulations can be long, scattered and difficult to follow. Students may miss deadlines not because they are careless, but because they do not know which regulation applies to them. Orbis reduces this problem by turning official regulation text into personalized, source-backed academic tasks.

This social impact is related to the United Nations Sustainable Development Goal 4, Quality Education, and Goal 10, Reduced Inequalities [1]. Students with less advising experience, double-major students, scholarship students, internship students and students near graduation can benefit from earlier warnings. However, wrong advice can also harm students. For this reason, Orbis should not replace advisors or Student Affairs. It should work as a support tool that shows the source document and allows human review when needed.

The environmental impact is smaller but still positive. A proactive digital advising system can reduce the need for printed handbooks, physical notices and repeated manual document distribution. The system also supports sustainability by caching embeddings and re-processing changed regulations only when needed. This avoids unnecessary repeated AI calls and keeps the computational cost lower.

Economically, Orbis can reduce repeated advising workload. In the evaluated run, the event pipeline processed 80 regulation sources, 935 regulation chunks and 323 candidate rules in 59 minutes, finally accepting 178 actionable obligations. Doing the same filtering manually would require much more human time. The project therefore has an economic benefit for the university by helping advisors and administrative units focus on more complex cases.

1.3 Project Outcomes

The expected outcome of the project is a working prototype that shows how institutional regulations can be transformed into explainable academic support tasks. The project outcome is not a fully autonomous advising authority. Instead, the outcome is a decision-support system that can surface relevant regulations, reduce information overload, and provide evidence for human review.

The measured outcome also shows where the design needs improvement. Event extraction reached high precision, but assignment matching was weaker and should remain reviewable. Therefore, the project outcome is useful as a prototype and as a foundation for future institutional deployment, but it requires stronger integration, broader evaluation and policy approval before production use.

1.4 Cost Analysis

The main cost of Orbis is labor. The software stack is mostly open source and the project was developed with existing student hardware. Still, the time spent on research, design, implementation, testing and reporting should be counted as a real engineering cost.

For this analysis, the hourly engineering labor cost is estimated as 300 TL/hour.

- Research, requirement analysis and planning: approximately 120 hours, 36,000 TL.
- Database design, architecture and data model work: approximately 110 hours, 33,000 TL.
- Knowledge-base crawling, taxonomy generation and RAG preparation: approximately 160 hours, 48,000 TL.
- Event pipeline, rule extraction and assignment matching: approximately 150 hours, 45,000 TL.
- Submission agent, appeal flow and safety tests: approximately 90 hours, 27,000 TL.
- Frontend/backend integration, debugging and user workflows: approximately 90 hours, 27,000 TL.

- Evaluation and reporting: approximately 150 hours, 45,000 TL.

The total labor estimate is approximately 870 hours. With 300 TL/hour, the labor cost becomes approximately 261,000 TL.

The technical cash cost is much lower:

- FastAPI, PostgreSQL, `pgvector`, React and Python libraries: 0 TL, because they are open-source tools.
- Existing development laptops: 0 TL additional project cost.
- Measured LLM evaluation cost: approximately 60 TL.
- Estimated cloud hosting for a small deployment: approximately 2,500 TL/month.
- Estimated three-month hosting cost: approximately 7,500 TL.
- Electricity and local development overhead: approximately 750 TL.

The approximate total research and development cost, including labor and technical costs, is therefore around 269,310 TL. If Orbis becomes a production system, the main continuing cost would be server hosting and maintenance rather than LLM usage.

1.5 Sustainability

The sustainability of Orbis can be discussed in three dimensions: educational sustainability, environmental sustainability and maintenance sustainability.

Educationally, Orbis supports sustainable academic advising because it reduces dependence on repeated one-to-one explanations of the same regulation. Once a regulation is crawled, structured and reviewed, it can support many students. This is related to SDG 4 because it improves access to academic information, and to SDG 10 because students with less personal advising support can still receive early warnings [1].

Environmentally, the project is lightweight. It does not require special hardware, mechanical components or continuous high-power computation. The design uses cached embeddings, stored source chunks and incremental reprocessing so that unchanged regulations do not need to be repeatedly embedded or analyzed. This reduces unnecessary model usage and supports a lower-compute deployment.

From a maintenance perspective, sustainability means that the system can survive regulation changes, semester changes and model changes. Orbis stores source references, timestamps, database records and evaluation data. Future maintainers can re-crawl official pages, compare changed documents, re-run evaluation cases and roll back a weaker prompt or model. This is important because a system that cannot be maintained safely would not be sustainable in a university setting.

1.6 Standards

In this section, the standards of engineering ethics, IEEE software engineering standards, IET professional conduct, EU regulations, Turkish regulations, information security and accessibility are discussed.

1.6.1 Engineering Code Of Conduct

The NSPE Code of Ethics states that engineers should hold public safety, health and welfare paramount [2]. The ACM/IEEE Software Engineering Code of Ethics also emphasizes public interest, client and employer responsibilities, product quality, judgment, management, profession, colleagues and self-improvement [3]. In Orbis, the main public welfare issue is student academic welfare. An incorrect notification or wrong submission decision can affect a student's academic progress.

For this reason, Orbis follows these ethical principles:

- Engineers shall avoid deceptive acts.
- Engineers shall acknowledge errors and should not hide known limitations.
- Engineers should act responsibly, ethically and lawfully.
- Systems that affect users should be explainable and reviewable.

In the project, these principles were applied by keeping source URLs, evidence text, audit logs and human appeal paths. The report also states the weak points of the system instead of presenting the prototype as fully complete. The IET Rules of Conduct are also relevant because they require professional conduct, integrity and responsible engineering behavior [4]. For Orbis, this means the system must communicate uncertainty and should not pretend to be an official university decision maker.

1.6.2 IEEE Standards

The following IEEE/ISO/IEC standards are related to Orbis:

- **IEEE/ISO/IEC 16085-2020 – Risk Management.** This standard is relevant because Orbis has risks such as wrong academic advice, missed obligations, stale regulations, privacy leakage and prompt injection [5].
- **ISO/IEC/IEEE 12207 – Software Life Cycle Processes.** This standard is related to the project’s software life cycle: requirements, design, implementation, testing, evaluation and maintenance [6].
- **ISO/IEC/IEEE 29148-2018 – Requirements Engineering.** Orbis converts student needs and university rules into explicit software requirements and atomic submission checks [7].
- **ISO/IEC/IEEE 24748-7000-2022 – Ethical Concerns During System Design.** This standard is relevant because Orbis uses AI in an educational decision-support setting [8].

1.6.3 Software Quality, Security and Accessibility Standards

Orbis should also be evaluated with general software quality and information security standards. ISO/IEC 25010 is relevant because it defines quality characteristics such as functional suitability, reliability, usability, security, maintainability and portability [9]. These characteristics map directly to the project: Orbis must return relevant academic information, remain available during advising periods, be understandable to students, protect private records, and remain maintainable when regulations change.

ISO/IEC 27001 is relevant for information security management because Orbis can process student profile fields and academic records [10]. A production deployment should include access control, least-privilege database users, audit logging, backup procedures and incident response. For the Turkish context, TS ISO/IEC 27001 certification guidance from the Turkish Standards Institution is also relevant for institutional information security practices [11].

The interface should follow WCAG 2.2 accessibility principles so that academic support is usable by students with different needs [12]. This requires readable text contrast, keyboard navigation, clear focus states, understandable error messages and accessible forms.

1.6.4 EU And Turkish Legal Standards

GDPR is relevant for European data-protection expectations [13], and KVKK Law No. 6698 is directly relevant for personal data processing in Turkey [14]. Since Orbis can process GPA status, scholarship information, internship status and course enrollment, only necessary student data should be used. Personal data should be processed for a clear academic-support purpose, retained only as long as needed, and protected with access control.

The EU Artificial Intelligence Act is also relevant because Orbis uses AI in an education-related setting [15]. Even though this project is a prototype, the design should follow the same risk-aware direction: human oversight, transparency, data governance, logging, evaluation and clear limitation notices. In practice, this means Orbis should provide source-backed outputs and should keep human advisors in the final decision loop.

2 Risks

In this section, the main project risks, potential outcomes, mitigation actions and backup plans are explained.

2.1 Risk Analysis

Risk analysis for Orbis focuses on risks that can affect students, advisors, university data and the reliability of academic support. Because Orbis is an AI-assisted system, the most important risks are incorrect academic advice, missed obligations, stale regulations, privacy leakage, prompt injection and low user trust. These risks and their backup plans are listed below.

2.2 Potential Risks and B Plan

- **Incorrect academic advice.** Potential outcome: a student receives an irrelevant or harmful obligation. Mitigation: show source evidence, confidence and review state for every assignment. B plan: route high-impact cases to an advisor before final use.
- **Missed obligation.** Potential outcome: a student does not receive a warning for a real requirement. Mitigation: keep the full regulation

searchable through RAG and evaluate recall. B plan: run a second recall-oriented extraction pass and advisor review.

- **Prompt injection in submissions.** Potential outcome: uploaded text tries to manipulate the submission agent. Mitigation: separate requirement decomposition from file checking and test malicious cases. B plan: send suspicious or low-confidence cases to human review.
- **Stale regulation.** Potential outcome: a changed rule remains outdated in the system. Mitigation: store source timestamps, content hashes and regulation snapshots. B plan: schedule re-crawls and keep semester-specific versions.
- **Privacy leakage.** Potential outcome: sensitive profile information is exposed unnecessarily. Mitigation: minimize prompt fields, restrict logs and apply access control. B plan: use anonymized profiles or an on-premise model.
- **SIS integration delay.** Potential outcome: the system cannot access official live student data. Mitigation: support synthetic profiles and manual import during prototype testing. B plan: keep Orbis as an advisory layer until official integration is approved.
- **Low user trust.** Potential outcome: students or advisors ignore the system. Mitigation: explain sources, uncertainty and limitations clearly. B plan: start with optional pilot usage and collect feedback.

2.3 Risk and Change Management

Risk management in Orbis follows the idea of identifying risks early, estimating their effect, applying mitigation and keeping a backup plan. The most important principle is that risky changes should be tested before they affect students. A new prompt, model, crawler rule or database migration should be evaluated on stored test cases and reviewed against previous results.

Regulation changes should trigger re-crawling and re-extraction. Prompt or model changes should be tested on the stored evaluation set before being used. Database changes should use migrations and backups. If a new model or prompt performs worse, the system should be rolled back to the previous stable version. For production, a change log should record the date, changed component, reason for change, evaluation result and rollback plan.

The project also needs human change management. Advisors and administrative units should know that Orbis is a support tool, not a replacement. Students should be told when a task is AI-generated and should have a way to request correction. This keeps the system aligned with ethical engineering practice and reduces the risk of over-trusting the prototype.

2.4 Challenges

In this section we can summarize the challenges faced during the project:

- **Data quality:** University web pages include regulations, announcements, event pages and duplicated bilingual content. Separating binding regulations from general information was difficult.
- **Rule ambiguity:** Some regulation sentences are formal but not directly assignable to a student. The system had to reject broad or non-actionable text.
- **Context matching:** Some obligations depend on GPA, internship state, scholarship state or graduation year. These cases are harder than simple keyword matching.
- **Evaluation:** There was no ready benchmark for Istanbul Bilgi University regulations. The gold labels had to be prepared manually.
- **Security:** The submission agent had to be tested against prompt-injection attempts inside uploaded documents.
- **Integration:** A full production version would require official SIS integration and institutional permission.

2.5 Weak And Strong Points

2.5.1 Weak Points

- The assignment-matching stage is the weakest measured subsystem. Its overall F_1 score is 0.51, which is acceptable for a prototype but not enough for unsupervised production decisions.
- The submission-agent test set is small. It includes 32 deterministic cases, so the result should not be interpreted as a complete security guarantee.

- The system currently uses synthetic or manually seeded student profiles instead of live SIS data.
- Stale regulations remain a risk unless scheduled re-crawling and regulation snapshots are used.
- Privacy and legal compliance would require a formal university deployment process.

2.5.2 Strong Points

- Orbis is traceable. Assignments and decisions are connected to source URLs, evidence text or reasoning logs.
- The event extraction stage has high precision. It reached 0.978 precision against the gold ground truth.
- The system is modular. The crawler, taxonomy system, event orchestrator, matcher, submission agent and frontend can be improved separately.
- The LLM cost is low compared to labor and hosting costs.
- The submission agent includes an appeal path, so rejected students can flag a case for human review.

3 Contributions

- **Author 1, Atakan Gül:** mainly worked on the EventPipelineOrchestrator, LLM reasoning agents, proactive event extraction, assignment matching, FastAPI backend integration, submission-review agent and safety/evaluation scripts.
- **Author 2, Arda Kaan Yıldız:** mainly worked on the Retrieval-Augmented Generation and taxonomy side of the project, including the institutional knowledge base, semantic discovery pipeline, URL/taxonomy classification and PostgreSQL/pgvector database architecture.
- **Supervisor, Özgür Özdemir:** supervised the project, gave feedback during the development process and guided the technical and reporting parts of the study.

Both authors also contributed to planning, debugging, manual labeling, evaluation, presentation preparation and report writing.

4 Conclusion

Orbis is an AI-supported academic advising project that aims to make university regulations more visible, personalized and actionable. The project has positive social and economic effects because it can reduce student confusion and administrative workload. It also supports sustainability by reducing repeated manual document handling and by using cached embeddings and controlled model calls.

The project also has important risks. Wrong academic advice, missed obligations, stale regulations, privacy leakage and prompt injection must be handled carefully. For this reason, Orbis should be used as an advisory and evidence-based support system, not as a final decision authority. Human review, source evidence, appeal paths and change management are necessary parts of the design.

As a result, the project follows the required engineering constraints, standards, risk analysis and contribution expectations. With stronger SIS integration, larger evaluation sets and formal deployment controls, Orbis can become a sustainable academic support tool for the university.

References

- [1] United Nations, “Transforming our world: The 2030 agenda for sustainable development.” <https://sdgs.un.org/2030agenda>, 2015.
- [2] National Society of Professional Engineers, “Nspe code of ethics for engineers.” <https://www.nspe.org/resources/ethics/code-ethics>, 2026.
- [3] ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices, “Software engineering code of ethics and professional practice.” <https://www.computer.org/education/code-of-ethics>, 1999.
- [4] The Institution of Engineering and Technology, “Rules of conduct.” <https://www.theiet.org/about/governance/rules-of-conduct>, 2024.
- [5] IEEE Standards Association, “IEEE/ISO/IEC 16085-2020: Systems and software engineering – life cycle processes – risk management.” <https://standards.ieee.org/ieee/16085/6932/>, 2020.
- [6] IEEE Standards Association, “IEEE/ISO/IEC 12207-2017: Systems and software engineering – software life cycle processes.” <https://standards.ieee.org/ieee/12207/5672/>, 2017.
- [7] IEEE Standards Association, “IEEE/ISO/IEC 29148-2018: Systems and software engineering – life cycle processes – requirements engineering.” <https://standards.ieee.org/standard/29148-2018.html>, 2018.
- [8] IEEE Standards Association, “IEEE/ISO/IEC 24748-7000-2022: Systems and software engineering – life cycle management – standard model process for addressing ethical concerns during system design.” <https://standards.ieee.org/>, 2022.
- [9] International Organization for Standardization, “ISO/IEC 25010:2011: Systems and software engineering – systems and software quality requirements and evaluation – system and software quality models.” <https://www.iso.org/standard/35733.html>, 2011.
- [10] International Organization for Standardization, “ISO/IEC 27001:2022: Information security, cybersecurity and privacy protection – information security management systems – requirements.” <https://www.iso.org/standard/27001>, 2022.

- [11] Türk Standardları Enstitüsü, “TS ISO/IEC 27001: Bilgi güvenliği yönetim sistemi belgelendirmesi.” <https://www.tse.org.tr/bilgi-guvenligi-yonetim-sistemi-bgys-belgelendirmesi-ts-iso-iec-27001/>, 2024.
- [12] World Wide Web Consortium, “Web content accessibility guidelines (WCAG) 2.2.” <https://www.w3.org/TR/WCAG22/>, 2023.
- [13] European Parliament and Council of the European Union, “Regulation (EU) 2016/679 of the european parliament and of the council.” <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng>, 2016.
- [14] Kişisel Verileri Koruma Kurumu, “Personal data protection law no. 6698.” <https://www.kvkk.gov.tr/Icerik/6649/Personal-Data-Protection-Law>, 2016.
- [15] European Parliament and Council of the European Union, “Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence.” <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>, 2024.